

Height Balanced Trees

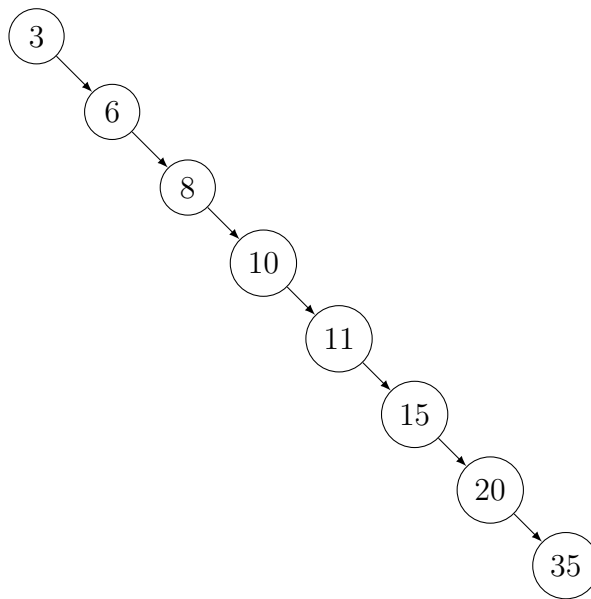
1 Skewed Binary Search Tree

Consider the keys

[3, 6, 8, 10, 11, 15, 20, 35]

inserted into a BST in the given order.

Since the keys are inserted in increasing order, the resulting BST is skewed to the right.



The height of this tree is $O(n)$, which makes searching, insertion, and deletion potentially take $O(n)$ time.

2 AVL Tree

An AVL tree is a self-balancing Binary Search Tree in which the heights of the left and right subtrees of every node differ by at most one.

The balance factor of a node is defined as

$$BF = h(\text{left subtree}) - h(\text{right subtree})$$

For an AVL tree,

$$BF \in \{-1, 0, +1\}$$

for every node.

If the balance factor becomes less than -1 or greater than $+1$, rotations are performed to restore the balance.

The four types of AVL rotations are:

1. Left Rotation
2. Right Rotation
3. Left-Right Rotation
4. Right-Left Rotation

An AVL tree has height

$$O(\log n)$$

Therefore, search, insertion, and deletion take

$$O(\log n)$$

time.

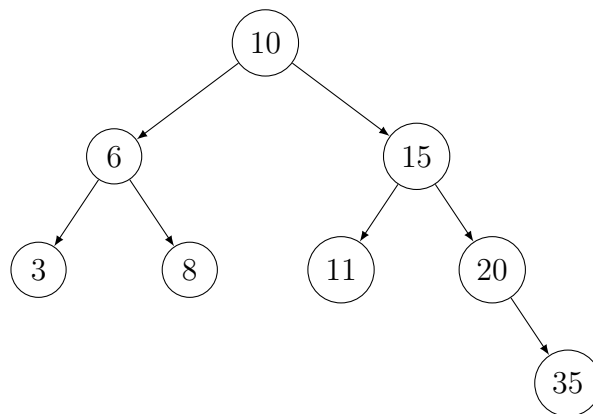
3 Height Adjustment of the Given BST

The original BST is highly skewed. To make it height balanced, AVL rotations are performed during insertion.

For the keys

$$[3, 6, 8, 10, 11, 15, 20, 35]$$

the resulting AVL tree is:



For every node in this tree, the balance factor is within

$$[-1, +1].$$

Therefore, the tree satisfies the AVL height-balance condition.

4 Red-Black Tree

A Red-Black Tree is a self-balancing Binary Search Tree in which every node has a color: red or black.

A Red-Black Tree satisfies the following conditions:

1. Every node is either red or black.
2. The root is black.
3. Every NIL leaf is black.
4. A red node cannot have a red child.
5. Every path from a node to its descendant NIL leaves contains the same number of black nodes.

These conditions ensure that the tree remains approximately balanced.

The height of a Red-Black Tree is

$$O(\log n).$$

Therefore, search, insertion, and deletion take

$$O(\log n)$$

time.

5 Deletion from the Height-Balanced Tree

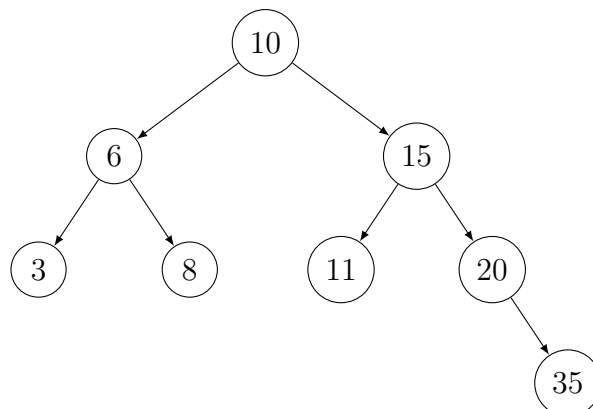
We now delete the keys

6, 11, 20

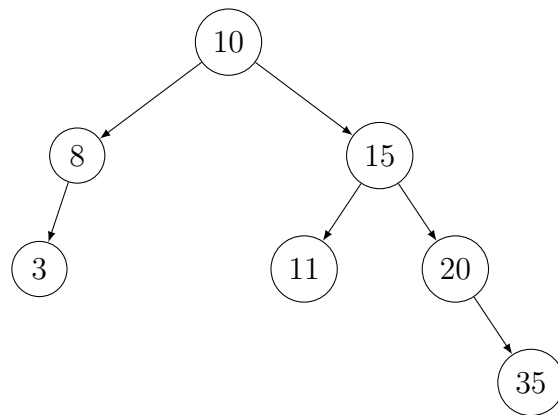
from the AVL tree while maintaining the AVL balance condition.

5.1 Deletion of 6

Before deletion:



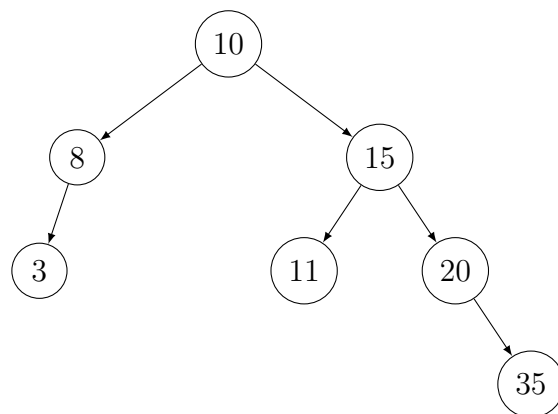
Node 6 has two children, 3 and 8.
Using the in-order successor, 8 replaces 6.
The resulting tree is:



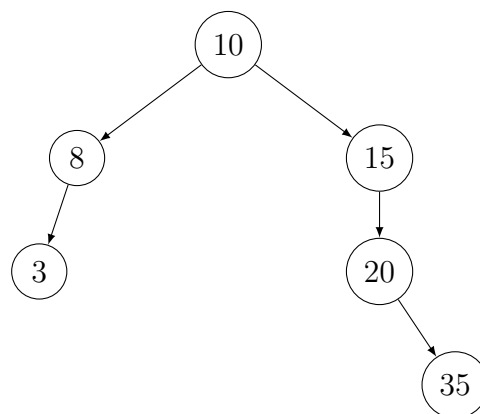
The balance factors remain within the required range, so no rotation is required.

5.2 Deletion of 11

Before deletion, the tree is:



Node 11 is a leaf, so it is simply removed.
The resulting tree is:



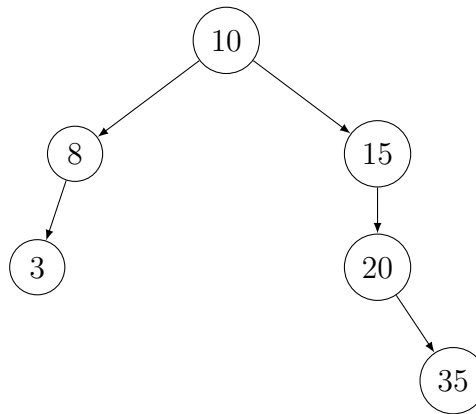
The balance factors remain within

$$[-1, +1],$$

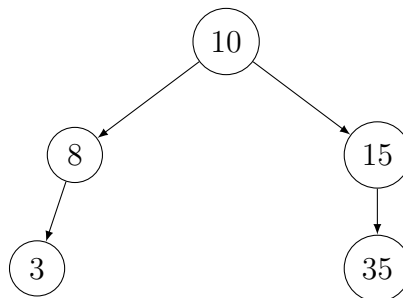
so no rotation is required.

5.3 Deletion of 20

Before deletion, the tree is:



Node 20 has one child, 35. Therefore, 35 replaces 20.
The resulting tree is:



The final AVL tree is balanced because every node has a balance factor in

$$\{-1, 0, +1\}.$$

The remaining keys are

$$[3, 8, 10, 15, 35].$$

6 B-Tree

A B-Tree is a self-balancing multiway search tree in which a node can contain multiple keys and multiple children.

For a B-Tree with minimum degree t :

- A node can contain at most $2t - 1$ keys.
- A node can have at most $2t$ children.

- Every non-root node contains at least $t - 1$ keys.
- An internal node containing k keys has $k + 1$ children.
- All leaf nodes are at the same level.
- Keys within a node are stored in sorted order.

B-Trees remain balanced after insertion and deletion.
The height of a B-Tree is

$$O(\log n).$$

B-Trees are commonly used in databases and file systems because a single node can contain many keys, reducing the number of disk accesses.

7 B+ Tree

A B+ Tree is a variation of the B-Tree designed for efficient storage and retrieval.

The main properties of a B+ Tree are:

- Internal nodes contain keys used for navigation.
- Actual records or data pointers are stored only in the leaf nodes.
- All leaf nodes are at the same level.
- Leaf nodes are linked together.
- Keys in the leaf nodes are stored in sorted order.

Since the leaf nodes are linked, sequential access and range queries can be performed efficiently.

The height of a B+ Tree is

$$O(\log n).$$

B+ Trees are widely used in database indexing and file systems.

8 B-Tree and B+ Tree Comparison

Feature	B-Tree	B+ Tree
Data storage	Data can be stored in internal and leaf nodes	Data is stored only in leaf nodes
Internal nodes	Store keys and possibly data	Store only keys for navigation
Leaf nodes	Not necessarily linked	Linked together
Sequential access	Less efficient	More efficient
Range queries	Efficient	Very efficient
Common use	Databases and file systems	Database indexing and file systems